Unit 4. Evaluation of ML systems

Machine Learning 1 (ML1)

Master in Artificial Intelligence (MIA)

November 13, 2025

4.1 Evaluation Metrics

We are going to focus on **classifiers** first.

To be trained, a model has its own function to quantify the **error** or **goodness of fit**.

E.g., Cross-entropy function in the case of neural networks.

But, once training is over, error may not necessarily be the best indicator.

- It is not significant.
- It depends on how outputs are coded.

There are specific evaluation criteria, usually comparing the **output** generated by the system with **correct predictions** on a given dataset:

- **Confusion matrix**.
- **Received Operating Characteristic** (ROC) curve $\Rightarrow$ **Area under the Curve** (AUC).
- Cohen's **kappa coefficient**.

4.1.1 Confusion matrix

A confusion matrix is a **square matrix**, with as many rows and columns as **classes**, showing the distribution of **instances** in classes, and the **classification** performed by the model.

In other words, it shows the distribution of **correct decisions** and **errors** made by a classifier along the different classes of the problem.

- Each **row** of the matrix represents the instances in an **actual class**.
- Each **column** represents the instances in a **predicted class**.

  (sometimes the other way around)
- It makes it easy to see whether the system is **confusing** two classes (mislabeling the instances in one class as belonging to another), hence the name confusion matrix.

For a **binary** classification problem, the confusion matrix has two classes in both dimensions: **positive** and **negative**.

		Prediction	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

It contains four values, which can be separated:

- according to the **output** of the model as **positive** or **negative**.
- depending on whether model **predictions** are **true** or **false**.

Let's see what they represent:

- **True Positive** (TP) is the number of **correct** predictions of examples as belonging to the **positive** class.

  E.g., in a spam filter, an e-mail correctly identified as spam.
- **False Negative** (FN) is the number of **incorrect** predictions of positive instances as members of the **negative** class.

  E.g., a spam e-mail being misclassified as non-spam.

  False negatives are also known as **Type II** errors, a term borrowed from statistical hypothesis testing.
- **False Positive** (FP) is the number of **incorrect** predictions of negative instances as members of the **positive** class.

  E.g., a non-spam e-mail being misclassified as spam.

  False positives ara also kown as **Type I** errors, again from statistical hypothesis testing.
- **True Negative** (TN) is the number of **correct** predictions of examples as belonging to the **negative** class.

  E.g., a non-spam e-mail being classified as such.

We can use the information in the confusion matrix to derive different **performance metrics**. Let's start with the obvious ones:

- **Accuracy**: The percentage of **correctly classified** examples.

$$Acc = \frac{TP + TN}{TP + TN + FP + FN} = 1 - ER$$

- **Error rate**: The percentage of **misclassified** examples.

$$ER = \frac{FP + FN}{TP + TN + FP + FN} = 1 - Acc$$

These measures are widely used, and give us all the information we need in many kinds of problems.

But in some cases, it may be of more interest to focus on situations in which a **positive response** is or should be produced by the model:

- Something **critical**, such as diagnosing a disease or raising an alarm.
- **Recall, Sensitivity or True Positive Rate**: the ratio of **correctly** classified **positive** examples, i.e. TP divided by all positive instances.

$$R = \frac{TP}{TP + FN}$$

		Prediction	
		Pos	Neg
Actual	Pos	**TP**	FN
	Neg		

It focus on examples in the **positive class only**.

- **Precision or Positive Predictive Value**: the ratio of true **positive** examples among all examples **predicted** as positive.

$$P = \frac{TP}{TP + FP}$$

		Prediction	
		Pos	Neg
Actual	Pos	**TP**	
	Neg	FP	

It focus **exclusively** on examples **predicted as positive**.

Recall and **precision** are commonly used **together** (more on that later).

- Having two different measures makes comparing the performance of different models challenging.
- If a model has higher precision but lower recall than another one, or viceversa, which one is better?

One way to avoid that conundrum is to **aggregate** precision and recall in a single measure:

- **F-measure**, **F-score**, **F$_1$-measure** or **F$_1$-score**: the **harmonic mean** of recall and precision.

  $$F = 2 \times \frac{P \times R}{P + R} = \frac{2}{P^{-1} + R^{-1}}$$

  The **range** of the F-measure is $[\mathbf{0}, \mathbf{1}]$, and its value is higher the higher that recall and precision are.

  If $R = P$ then $R = P = F$.

  As a **harmonic** mean, it is always **lower or equal** than the **arithmetic mean** of recall and precision $\Rightarrow$ a more **conservative** estimate.

  The subscript 1 is used because the F-measure is a particular case of the F$_\beta$-measure with $\beta = 1$:

  $$F_\beta = (1 + \beta^2) \times \frac{P \times R}{\beta^2 \times P + R}$$

Recall and precision focus on **positive instances**, but for some applications we may be interested on how **negative examples** are classified.

- **Specificity, Selectivity or True Negative Rate**: the ratio of **correctly** classified **negative** examples, i.e. TN divided by all negative instances.

  $$S = \frac{TN}{FP + TN}$$

		Prediction	
		Pos	Neg
Actual	Pos		
	Neg	FP	**TN**

  It focus on **negative instances only**.

- **Negative Predictive Value**: the ratio of true **negative** examples among all examples **predicted** as negative.

  $$NPV = \frac{TN}{TN + FN}$$

		Prediction	
		Pos	Neg
Actual	Pos		FN
	Neg		**TN**

  It focus exclusively on **examples predicted as negative**.

Accuracy and its pitfalls

Let's start with an example: we have a patient database of 1000 cases:

- 5 of them **sick** (**positive** class).
- 995 **healthy** (**negative** class).

If a model classifies all of them as **healthy**:

		Prediction	
		Positive	Negative
Actual	Positive	0	5
	Negative	0	995

$$Acc = \frac{0 + 995}{0 + 995 + 0 + 5} = 0.995\ (99.5\%) \qquad ER = 1 - Acc = 0.005\ (0.5\%)$$

Very **simple classifier**, with very **good accuracy**.

The **problem** here is **unbalanced data**:

- **Negative** instances (healthy patients) far **outweight** positive ones (sick people).
- A classifier that labels all patients as healthy will have good accuracy.
- But we want to find if people are sick!
- **In general, accuracy (and error rate) do not work very well with unbalanced data**.

In **medical testing**, instead of focusing on accuracy, it is standard to check **sensitivity** (recall) and **specificity**.

$$R = \frac{0}{0 + 5} = 0\ (0\%) \qquad S = \frac{995}{0 + 995} = 1\ (100\%)$$

- All positive cases have been misclassified $\Rightarrow$ **all false positives**.
- All negative cases have been accurately classified $\Rightarrow$ **no false negatives**.

Having both metrics is more informative than just accuracy (or error rate).

Recall and Precision

Many times, **positive** instances are **more important** than negative ones:

- Alarm systems.
- Medical diagnosis.
- Detection of important objects in images or signals.

  E.g., detection of arrhythmia in an ECG.

  E.g., detection of a pedestrian in autonomous driving.

Metrics that **focus on positive** cases make sense for this kind of problems:

- **Recall** represents **completeness** $\Rightarrow$ what percentage of positive instances are labeled as such.

  E.g., how many alarms are correctly raised, how many cancer cases are correctly diagnosed.
- **Precision** can be thought of as **exactness** $\Rightarrow$ what percentage of instances labeled as positive are actually so.

  E.g., how many alarms raised are justified, how many cancer diagnosis are, in fact, cancer.

On the other hand, it is often possible to construct **baseline models** that maximize one metric but not the other.

- A model that declares **every instance** to be the **positive** will have a **perfect recall**.
  - If the number of negative examples in the data is substantial, **precision** will be **poor** $\Rightarrow$ many false positives.
    E.g., clasifying all patients as healthy in a medical test.
- Conversely, a model that only labels as positive a **reduced number** of instances **sure to be positive** will have **perfect precision**.
  - If the number of positive examples in the data is substantial, **recall** will be **poor** $\Rightarrow$ many false negatives.
    E.g., in autonomous driving, a stop sign detector the only works in perfect conditions: good light, high visibility, no rain/fog, ...

That it is not necessarily a problem if the **focus** of our model is **minimizing** an especific kind of error:

- If minimizing **false negatives** is our goal $\Rightarrow$ focus on **recall**.
- If we want to minimize **false positives** above all $\Rightarrow$ focus on **precision**.

E.g., usually, in **medical diagnosis**:

- The cost of misdiagnosing a sick patient as healthy is very high:
  - The patient could die or be left severely impaired.
  - We want to **maximize recall**.
- The cost of misdiagnosing a healthy patient as sick is usually lower:
  - More tests will be made $\Rightarrow$ good chance of catching the error.
  - **Precision** is **less important**.

But in many applications we want to **maximize both precision and recall** $\Rightarrow$ A **key challenge** for many classification problems and algorithms.

In the end, the metric to rely on depends on the **kind of problem** we are trying to solve, and especially in the **cost of erroneous predictions**.

E.g., an alarm system:

- Situations in which an alarm should be raised are the positive examples.
- Situations in which an alarm is raised are the positive predictions.

		Prediction	
		Positive	Negative
Actual	Positive	48	6
	Negative	3	837

$$Acc = \frac{48 + 837}{48 + 6 + 3 + 837} = 0.9899\ (98.99\%)$$

$$R = \frac{48}{48 + 6} = 0.8889\ (88.89\%) \quad P = \frac{48}{48 + 3} = 0.9412\ (94.12\%)$$

- In 6 of 54 cases (11.11%) an alarm was not raised when it should have.
- 3 of 51 (5.88%) were false alarms.

Which metric is more valuable? Are these acceptable results?

- It is an 11.11% of alarms not raised a reasonable risk?
- It is a 5.88% of false alarms an acceptable nuisance?

Again, it depends on **costs**:

- If the cost of an **alarm not raised** is high $\Rightarrow$ try to maximize **recall**.
- If the cost of a **false alarm** is high $\Rightarrow$ try to maximize **precision**.

We use...	...when the system has to be...	...risking...
Recall	very good in the detection of situations in which an alarm is warranted	more false alarms
Precision	very sure that an alarm has to be raised	more alarms not raised that should have been
Specificity	very good in the detection of situations in which an alarm is not warranted	more alarms not raised that should have been
Negative Predictive Value	very sure that an alarm is not to be raised	more false alarms

Sensitivity (recall) and specificity

Looking at the previous example we can see that **sensitivity** and **specificity** have the **same** kind of **trade offs** that **recall** and **precision**.

- **Higher sensitivities** will mean **lower specificities** and vice versa.
- If minimizing **false negatives** is our goal $\Rightarrow$ focus on **sensitivity**.
- If we want to minimize **false positives** above all $\Rightarrow$ focus on **specificity**.

E.g., again **medical diagnosis**:

- If the consequence of failing to treat the condition is serious.
  - Minimize **false negatives** on test $\Rightarrow$ focus on **sensitivity**.
- If the consequences of not treating it are not serious, but those of misdiagnosing healthy patients are (expense, cultural stigma, ...):
  - Minimize **false positives** on test $\Rightarrow$ focus on **specificity**.

Accuracy vs F-measure

A possible **rule of thumb** is to use **accuracy** when the number of instances among classes is similar, i.e. the dataset is **balanced**, and **F-score otherwise**.

- **Recall** and **precision** are more appropriate with **unbalanced** datasets.
- F-score is an **aggregation** of both.
- **Unbalanced** datasets are **typical**, so **F-score** tends to be a better choice in many cases.

Another rule of thumb is to use **accuracy** when **true positives** and **true negatives** are more important, and **F-score** is used when **false negatives** and **false positive**s are crucial.

Accuracy

		Prediction	
		Pos	Neg
Actual	Pos	TP	
	Neg		TN

F-score

		Prediction	
		Pos	Neg
Actual	Pos		FN
	Neg	FP	

The **harmonic mean** is used in the F-measure because it **penalizes extreme values** better than the arithmetic mean.

Example:

- Balanced classes (200 instances in each one).
- Classifier predicts the negative class in most of the instances.

		Prediction	
		Positive	Negative
Actual	Positive	5	195
	Negative	2	198

$$Acc = \frac{5 + 198}{5 + 198 + 2 + 195} = 0.5075\ (50.75\%)$$

$$R = \frac{5}{5 + 195} = 0.025\ (2.5\%) \quad P = \frac{5}{5 + 2} = 0.7142\ (71.42\%)$$

$$F = 2 \times \frac{0.7142 \times 0.025}{0.7142 + 0.025} = 0.0483\ (4.83\%)$$

While accuracy is not good (50.75%), F-measure is extremely low (4.83%).

- Due to the low recall value.
- F-measure gives low scores to low precision or recall.
- Only when **both recall and precision are high, F-measure is high**.

Example: same dataset, two classifiers.

		Prediction	
		Pos	Neg
Actual	Pos	160	40
	Neg	40	160

		Prediction	
		Pos	Neg
Actual	Pos	200	0
	Neg	133	67

$$Acc = 0.8\ \ (80\%)$$
$$R = 0.8\ \ (80\%)$$
$$P = 0.8\ \ (80\%)$$
$$F = 0.8\ \ (80\%)$$

$$Acc = 0.6675\ \ (66.75\%)$$
$$R = 1\ \ (100\%)$$
$$P = 0.6006\ \ (60.06)\%$$
$$F = 0.7505\ \ (75.05)\%$$

The classifier on the left has lower recall but higher precision that the one on the right.

- But **precision** and **recall** are **more similar** (in fact the same, 80%)
- So its **F-measure** is **higher** (80%) than the one for the righmost classifier (75.05%).
- If we used an arithmetic mean instead of a harmonic one, both classifiers would have aproximately the same F-measure (80%).
  - The **harmonic mean** is **more conservative** (pessimistic) than the arithmetic mean.

Some authors critizice the **F-score because** it gives **equal weight** to recall and precision.

- Recall may be more important for a given problem, o viceversa.
- They propose the **F$_\beta$-measure** instead, with β chosen for each problem.

Multiclass classifiers

We can extend the concept of **confusion matrix** to multiclass problems.

- Instead of just two rows/columns for the positive and negative class,
- $\boldsymbol{K}$ **rows/columns**: one per class.

		Prediction			
		Class 1	Class 2	$\ldots$	Class K
Actual	Class 1	C_{11}	C_{12}	$\ldots$	C_{1K}
	Class 2	C_{21}	C_{22}	$\ldots$	C_{2K}
	$\vdots$				
	Class K	C_{K1}	C_{K2}	$\ldots$	C_{KK}

- $\boldsymbol{C_{ij}}$ represents the number of **instances** in **class** $\boldsymbol{i}$ **predicted** as members of **class** $\boldsymbol{j}$.

As in the binary case, the confusion matrix gives us a representation of the **classifier behaviour**.

- which can be used to **guide** further **development** of the model.

E.g., three classes A, B and C, and 23 instances of each class.

		Prediction		
		A	B	C
Actual	A	20	1	2
	B	3	11	9
	C	1	10	12

- 10 class C instances have been labeled as class B.
- 9 class B instances have been labeled as class C.
- The classifier does not separate well classes B and C.
- We can try to ameliorate this problem.

  E.g., finding new features that help the classifier to distinguish between classes B and C.

Regarding metrics, we can generalize **accuracy** and **error rate**, computing them **directly** from the values of the confusion matrix:

$$Acc = \frac{\sum_{i=1}^{K} C_{ii}}{\sum_{k=1}^{K} \sum_{l=1}^{K} C_{kl}} \qquad ER = 1 - Acc = \frac{\sum_{i=1}^{K} \sum_{j \neq i} C_{ij}}{\sum_{k=1}^{K} \sum_{l=1}^{K} C_{kl}}$$

- $\boldsymbol{C_{ii}}$: **correctly classified** examples for class i.
- $\boldsymbol{C_{ij}, \ i \neq j}$: instances of class i **misclassified** as members of class j.

Other metrics like **recall**, **precision**, **F-measure**, **specificity** and **negative predictive value** can be adapted to multiclass classification:

- We treat the K-class problem as $\boldsymbol{K}$ **binary problems**, in a **one-vs-all** manner.

  In each binary problem, one class is separated from the rest.
- We compute TP_i, TN_i, FP_i and FN_i for each class i against the rest.
- We can compute **recall**, **precision** and **F-measure** for **each class** $\boldsymbol{i}$.

$$R_i = \frac{TP_i}{TP_i + FN_i} \quad P_i = \frac{TP_i}{TP_i + FP_i} \quad F_i = 2 \times \frac{P_i \times R_i}{P_i + R_i}$$

Example:

		Prediction		
		A	B	C
Actual	A	8	5	20
	B	2	10	10
	C	5	5	70

$$Acc = \frac{88}{135} = 0.6519 \ (65.19\%)$$

$$ER = \frac{47}{135} = 0.3481 \ (34.81\%)$$

		Prediction	
		A	¬A
Act.	A	8	25
	¬A	7	95

		Prediction	
		B	¬B
Act.	B	10	12
	¬B	10	103

		Prediction	
		C	¬C
Act.	C	70	10
	¬C	30	25

Problem: what to do with $\boldsymbol{K}$ **values** of recall, precision and F-measure.

- We need to **aggregate** them into a single value.
- Three ways to do it: **macro**, **weighted** and **micro** averaged.

Macro-averaged: the global value of the metric will be the **arithmetic mean** of the K values computed for the classes.

$$R_M = \frac{\sum_{i=1}^{K} R_i}{K} \qquad P_M = \frac{\sum_{i=1}^{K} P_i}{K} \qquad F_M = \frac{\sum_{i=1}^{K} F_i}{K}$$

$\boldsymbol{R_M}$ is the macro-averaged **recall**, or macro recall

$\boldsymbol{P_M}$ is the macro-averaged **precision**, or macro precision

$\boldsymbol{F_M}$ is the macro-averaged **F-score**, or macro F_1.

- This method does not take into account if classes are **unbalanced**.

Weighted-averaged: the global value of the metric will be the **weighted arithmetic mean** of the K values computed for the classes.

$$R_W = \sum_{i=1}^{K} w_i \, R_i \qquad P_W = \sum_{i=1}^{K} w_i \, P_i \qquad F_W = \sum_{i=1}^{K} w_i \, F_i$$

$$\text{with} \quad w_i = \frac{\text{instances in class } i}{\text{all instances}} = \frac{\sum_{j=1}^{K} C_{ij}}{\sum_{k=1}^{K} \sum_{l=1}^{K} C_{kl}}$$

$\boldsymbol{R_W}$ is the weighted-averaged **recall**, or weighted recall

$\boldsymbol{P_W}$ is the weighted-averaged **precision**, or weighted precision

$\boldsymbol{F_W}$ is the weighted-averaged **F-score**, or weighted F_1.

- More suitable than macro-averaged for **unbalanced** datasets.
  - It takes into account the **relative size** of each class.

Micro-averaged: TP, FP and FN are calculated globally from the values for each class.

$$R_\mu = \frac{\sum_{i=1}^{K} TP_i}{\sum_{i=1}^{K} (TP_i + FN_i)} \qquad P_\mu = \frac{\sum_{i=1}^{K} TP_i}{\sum_{i=1}^{K} (TP_i + FP_i)} \qquad F_\mu = 2 \times \frac{P_\mu \times R_\mu}{P_\mu + R_\mu}$$

$\boldsymbol{R_\mu}$ is the micro-averaged **recall**, or micro recall

$\boldsymbol{P_\mu}$ is the micro-averaged **precision**, or micro precision

$\boldsymbol{F_\mu}$ is the micro-averaged **F-score**, or micro F_1.

- **Important**: $R_\mu = P_\mu = F_\mu = Acc$ if classes are **mutually exclusive**.
- Only used when classes are **not mutually exclusive** and the dataset is **not unbalanced** (or unbalancing is not important).

Example: from the previous binary problems:

		Prediction	
		A	¬A
Act.	A	8	25
	¬A	7	95

		Prediction	
		B	¬B
Act.	B	10	12
	¬B	10	103

		Prediction	
		C	¬C
Act.	C	70	10
	¬C	30	25

$$R_A = \frac{8}{8+25} = 0.2424 \qquad R_B = \frac{10}{10+12} = 0.4545 \qquad R_C = \frac{70}{70+10} = 0.875$$

$$P_A = \frac{8}{8+7} = 0.5333 \qquad P_B = \frac{10}{10+10} = 0.5 \qquad P_C = \frac{70}{70+30} = 0.7$$

$$F_A = 2 \times \frac{0.5333 \times 0.2424}{0.5333 + 0.2424} = 0.3333 \qquad F_C = 2 \times \frac{0.5 \times 0.4545}{0.5 + 0.4545} = 0.5555 \qquad F_C = 2 \times \frac{0.7 \times 0.8759}{0.7 + 0.875} = 0.4762$$

- macro-averaged metrics:

$$R_M = \frac{0.2424 + 0.4545 + 0.875}{3} = 0.524\ (52.40\%)$$

$$P_M = \frac{0.5333 + 0.5 + 0.7}{3} = 0.5778\ (57.70\%)$$

$$F_M = \frac{0.3333 + 0.5555 + 0.4762}{3} = 0.4550\ (45.50\%)$$

- weighted-averaged metrics:

$$R_W = \frac{33}{135} \times 0.2424 + \frac{22}{135} \times 0.4545 + \frac{80}{135} \times 0.875 = 0.6518\ (65.18\%)$$

$$P_W = \frac{33}{135} \times 0.5333 + \frac{22}{135} \times 0.5 + \frac{80}{135} \times 0.7 = 0.6266\ (62.66\%)$$

$$F_W = \frac{33}{135} \times 0.3333 + \frac{22}{135} \times 0.5555 + \frac{80}{135} \times 0.4762 = 0.4542\ (45.42\%)$$

- micro-averaged metrics:

		Prediction	
		Pos	Neg
Actual	Pos	88	47
	Neg	47	223

$$R_\mu = P_\mu = \frac{88}{88+47} = 0.6519\ (65.19\%)$$

$$F_\mu = 2 \times \frac{0.6519 \times 0.6519}{0.6519 + 0.6519} = 0.6519\ (65.19\%)$$

4.1.2 Receiver Operating Characteristic (ROC) curve

Binary Classifiers

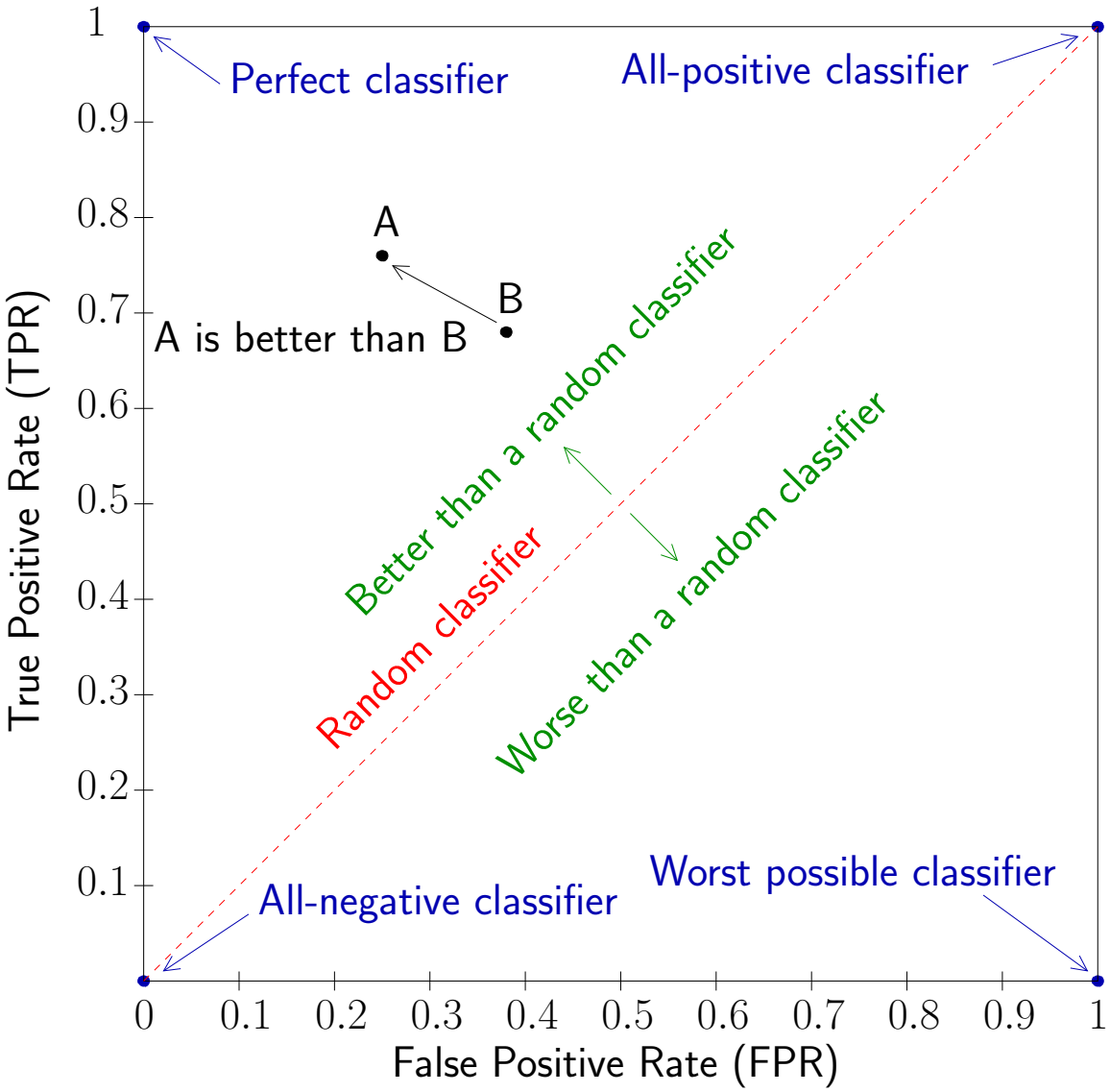
ROC space is a **two-dimensional** space with the false positive rate on the horizontal axis and the true positive rate on the vertical axis:

- **False Positive Rate**: how badly negative samples are classified.
  Equal to $1-$ specificity.
  $$FPR = 1 - S = 1 - \frac{TN}{FP+TN} = \frac{FP}{FP+TN}$$
- **True Positive Rate**: how well positive samples are classified.
  It is the recall or sensitivity.
  $$TPR = R = \frac{TP}{TP+FN}$$
- **Range** in both axis is $[\mathbf{0}, \mathbf{1}]$.

A **point** with **coordinates** $(\boldsymbol{x}, \boldsymbol{y})$ in the ROC space represents a classifier with **FPR** $\boldsymbol{x}$ and **TPR** $\boldsymbol{y}$.

- **Point** $(\mathbf{1}, \mathbf{0})$ represents the **worst** possible classifier.
  - Highest possible FPR: $TN = 0 \Rightarrow$ all negative instances misclassified.
  - Lowest possible TPR: $TP = 0 \Rightarrow$ all positive instances misclassified.
- **Point** $(\mathbf{0}, \mathbf{1})$ represents a **perfect** classifier.
  - Lowest possible FPR: $FP = 0 \Rightarrow$ no negative instances misclassified.
  - Highest possible TPR: $FN = 0 \Rightarrow$ no positive instances misclassified.

- **Point $(0, 0)$**: classifier that predicts all instances in the **negative** class.
  - Lowest possible FPR: $FP = 0 \Rightarrow$ no negative instances misclassified.
  - Lowest possible TPR: $TP = 0 \Rightarrow$ all positive instances misclassified.
- **Point $(1, 1)$**: classifier that predicts all instances in the **positive** class.
  - Highest possible FPR: $TN = 0 \Rightarrow$ all negative instances misclassified.
  - Highest possible TPR: $FN = 0 \Rightarrow$ no positive instances misclassified.
- The further **northwest** in ROC space a classifier is located, the **better** its performance.

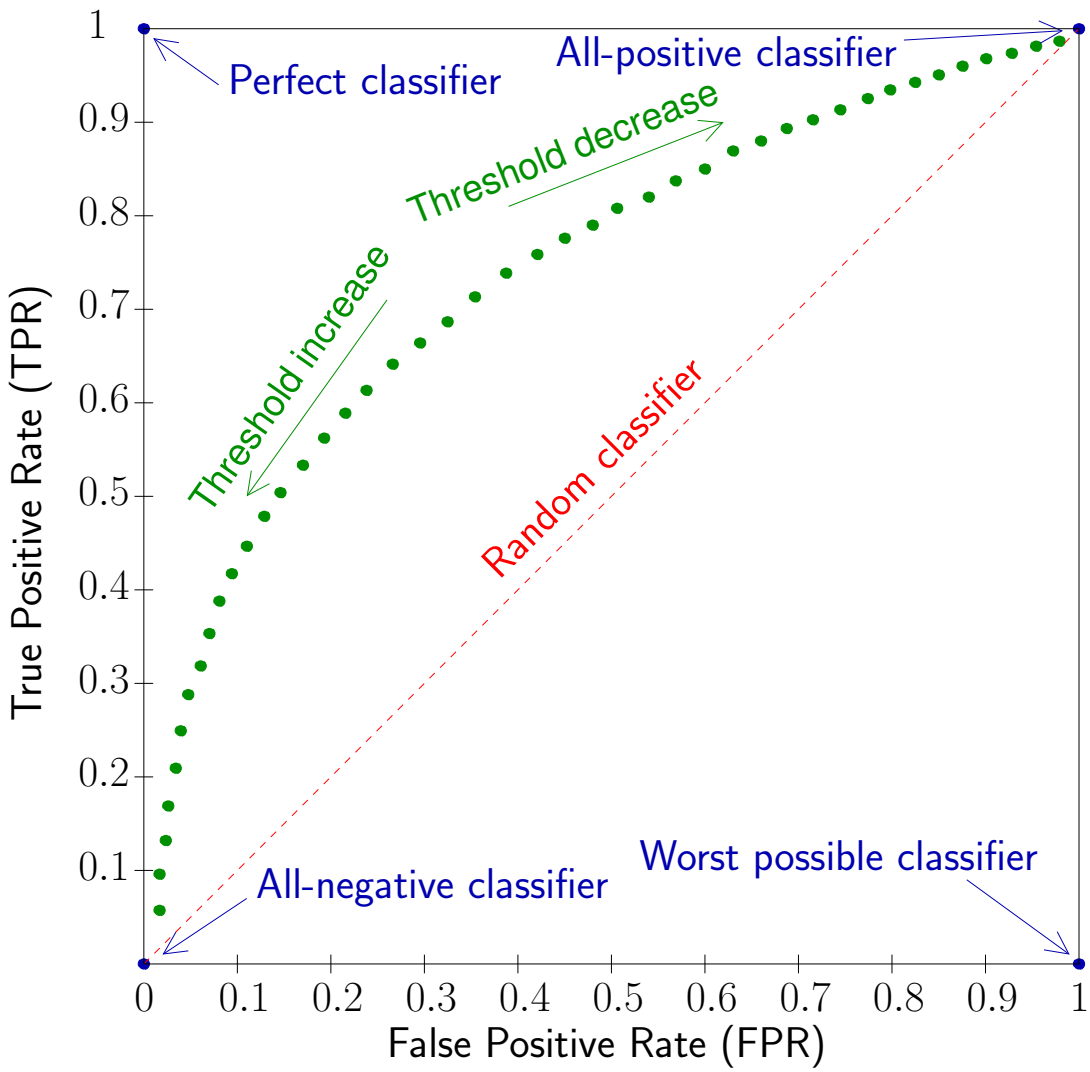


In most learning algorithms there is some **parameter** that can be changed:

- **Increase true positives** at the expense of increasing **false positives**.
- **Decrease false negatives** at the expense of decreasing **true negatives**.

E.g., the **threshold** of the **output neuron** in a ANN trained for **binary classification**.

- The unit outputs a value between 0 and 1.
- Given a threshold (usually 0.5).
  - outputs **lower** than the threshold $\Rightarrow$ **negative** class.
  - outputs **higher** than the threshold $\Rightarrow$ **positive** class.
- **Increase** the threshold $\Rightarrow$ more instances classified as **negative**.
  - TN and FN increase.
- **Decrease** the threshold $\Rightarrow$ more instances classified as **positive**.
  - TP and FP increase.
- Each threshold value puts the **classifier** in different **coordinates** of ROC space.
- The **ROC curve** represents **all points** for values of this parameter.



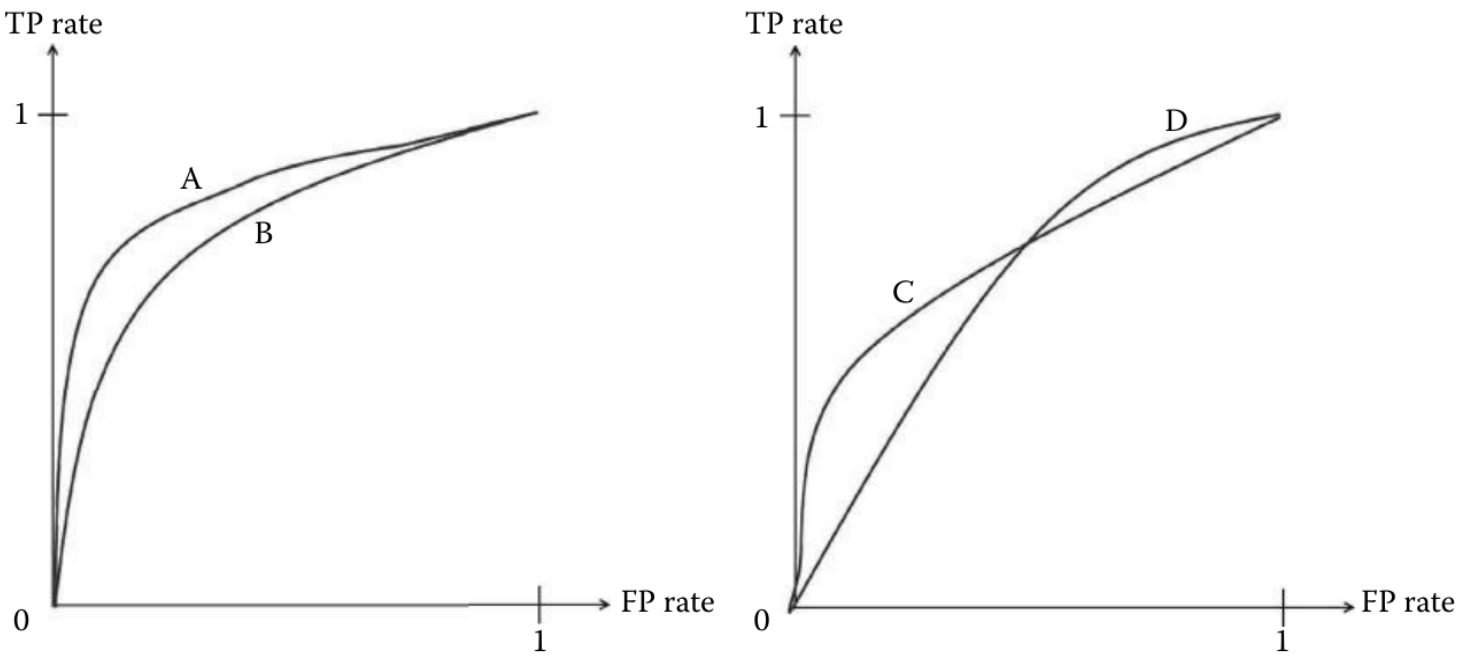
This treatment can be used on any kind of classifer with **probabilities** as **outputs**.

- ROC curves are a good tool to **visualize** and analyze the **performance** of learning algorithms.
- They are **independent of the class distribution**, which makes them interesting to evaluate **unbalanced** problems.

In order to **compare** two kinds of probabilistic classifiers, their **ROC curves** can be plotted on the **same ROC space**.

- In general, the **nearer** the ROC curve is to the coordinates $(0, 1)$, the **better** the classifier.

But it is not always easy to compare them:



Source: "Data classification Algorithms and Applications" - Aggarwal et al

In the left graph is clear that classifier A outperforms B.

- Not so easy to have a judgement in the right graph.
- But interesting information can still be extracted.
  - C is more precise than D for high thresholds, so it is the best choice if not many false positives are allowed.
  - D is more appropriate when a high true positive rate (recall) is needed.

Another possibility is to get a **single value** to assess performance.

- **Area Under the Curve** (AUC) is often used.
- AUC is the **area** between the **ROC curve** and the **horizontal axis**.
  - AUC ≤ 1.
  - Good measure to reflect classifier performance, as ROC curves in a **more northwesterly** location have a **bigger AUC**.
  - Often used as a single evaluation metric in **unbalanced** classification problems $\Rightarrow$ more informative than accuracy.

Multiclass classifiers

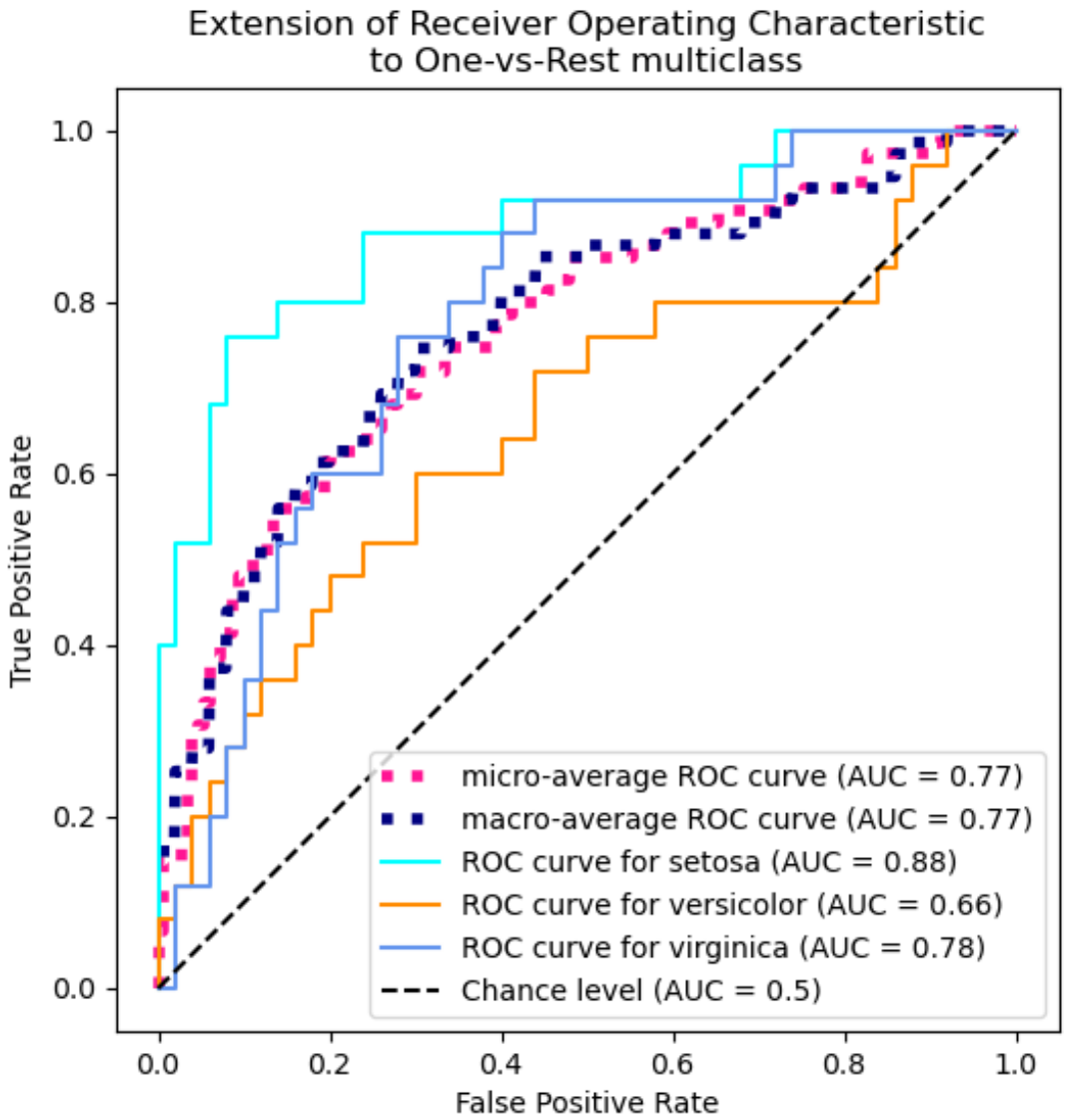
ROC curves and AUCs can be extended to **multiclass** classifers.

- **One-vs-Rest** (OvR): with K classes we can generate K binary problems in a **one-vs-all** manner:
  - One class i is separated from the rest.
  - We compute FPR, TPR, and ROC curve for class i against the rest.
- **One-vs-One** (OvO): we generate $K \times (K-1)/2$ binary problems, one for **each possible pair** of classes.
  - The first class of the pair will be the positive class, and the second the negative class.
  - We compute FPR, TPR, and ROC curves for each pair of classes.
- We can analize the ROC curve and AUC for **each** binary problem.

A common practice is to **aggregate** the ROC curves, usually in the OvR scheme.

- **Macro-average**: FPR and TPR computed for each class are averaged.
- **Weighted-average**: FPR and TPR computed for each class are averaged, with each class weighted in proportion of the number of instances belonging to that class.
- **Micro-average**: global FPR and TPR are computed from all the instances in all classes.

E.g., multinomial logistic regression on the iris dataset (scikit-learn.org).



4.1.3 Cohen's kappa coefficient

Proposed by Jacob Cohen to measure **agreement** between **two raters**.

- Later generalized by Fleiss to more than two.

It can be used to measure **agreement** between **two classifiers**.

- How much their outputs match.

It can also be used to measure **agreement** between a **classifier** and **real outputs** $\Rightarrow$ what we are interested here.

The calculation relies on the contents of a **matrix** reflecting the number of **agreements** and **disagreements**, similar to a confusion matrix.

		Method B		
		Positive	Negative	Total
Method A	Positive	a	b	$r_{pos} = a + b$
	Negative	c	d	$r_{neg} = c + d$
	Total	$c_{pos} = a + c$	$c_{neg} = b + d$	$n = a + b + c + d$

- If one of the methods is the observed outputs, it is the confusion matrix.
- **Agreement** can be measured as the ratio between the number of **matching outputs** and **all outputs**.

$$p_o = \frac{a + d}{n}$$

But, even if there is no relationship between both classifiers, some **agreement** can be expected due to **randomness**.

E.g., if both raters rely in coin flipping we can expect an average agreement of 50%.

We can even find some agreement between:

- a classifier and random choice.
- the desired outputs and random choice.

Cohen's kappa measures if the **observed agreement** is **higher** than what can be expected from **chance**.

For that, first the probability of **random agreement** has to be determined:

- The probability of classifying an instance in the **positive** class is:

$$\text{Method A: } \frac{r_{pos}}{n} \qquad \text{Method B: } \frac{c_{pos}}{n}$$

- The probability of classifying an instance in the **negative** class is:

$$\text{Method A: } \frac{r_{neg}}{n} \qquad \text{Method B: } \frac{c_{neg}}{n}$$

- If both methods (A and B) are **independent** the probability of they **agreeing** in the classification of an instance as positive **at random** is the product of both probabilities:

$$p_{pos} = \frac{r_{pos}}{n} \times \frac{c_{pos}}{n} = \frac{r_{pos} \times c_{pos}}{n^2}$$

- And the probability of agreement at random on a negative instance is:

$$p_{neg} = \frac{r_{neg}}{n} \times \frac{c_{neg}}{n} = \frac{r_{neg} \times c_{neg}}{n^2}$$

- So the probability of **agreement** due to **random chance** will be:

$$p_e = p_{pos} + p_{neg} = \frac{r_{pos} \times c_{pos} + r_{neg} \times c_{neg}}{n^2}$$

If p_e is the **expected** agreement due to random chance, then $1 - p_e$ is the margin of possible agreement non atributable to chance.

On the other hand, p_o is the **observed agreement**.

- If agreement is **complete** $\Rightarrow p_o = 1$.
- If one of the methods is the **desired output** $\Rightarrow p_o = Acc$.

So $p_o - p_e$ will be the amount of observed agreement **not** attributable to **random chance**.

Cohen's kappa:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

- **Ratio** between:
  - **observed agreement** not attributable to chance.
  - **margin of agreement** not attributable to chance.
- $p_o \leq 1 \Rightarrow p_o - p_e \leq 1 - p_e \Rightarrow \boldsymbol{\kappa \leq 1}$

- $\boldsymbol{\kappa = 1}$ when $p_o = 1$, i.e. when the observed agreement is **complete**.
- $\boldsymbol{\kappa = 0}$ when $p_o = p_e$, i.e. when the observed agreement is **equal** to the agreement due to chance.
- $\boldsymbol{\kappa < 0}$ when $p_o < p_e$, i.e. when the observed agreement is **lower** than the one expected by chance.
- A **guideline** to assess κ values was proposed by Landis & Koch:

κ	Degree of agreement
< 0	No agreement
$[0, 0.2]$	Slight
$(0.2, 0.4]$	Fair
$(0.4, 0.6]$	Moderate
$(0.6, 0.8]$	Substantial
$(0.8, 1]$	Almost perfect

  Based on personal experience $\Rightarrow$ more a **heuristic** than a hard rule.

Cohen's kappa can give a **better** assessment than **accuracy** when dealing with **unbalanced** data, but:

- More **dificult to interpret** than accuracy:
  - It says little of the **expected accuracy** of future predictions.
  - Still useful to **compare** or **optimize** models.
- Its value, even using the same model, is **higher when the test dataset is balanced**.

Finally, it can be generalized to:

- **Multiclass classifiers**:

	1	2	...	K	Total
1	C_{11}	C_{12}	...	C_{1K}	f_1
2	C_{21}	C_{22}	...	C_{2K}	f_2
		$\vdots$			
K	C_{K1}	C_{K2}	...	C_{KK}	f_K
	c_1	c_2		c_K	n

$$p_e = \sum_{i=1}^{K} \frac{f_i \times c_i}{n^2}$$

- **More** than **two raters** $\Rightarrow$ **Fleiss kappa**.

4.1.4 Regression Metrics

Some metrics used in model evaluation for regression problems.

Mean Squared Error (MSE): arithmetic **mean** of the **Residual Sum of Squares** or RSS (squared differences between real and predicted values).

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

where y_i is the real value, $\hat{y}_i$ the predicted one, $y_i - \hat{y}_i$ is a **residual**, and n is the number of observations.

- Most **commonly** used measure.
- Values between $\mathbf{0}$ **and** $\boldsymbol{\infty}$.
  - The **closer** it is to $\mathbf{0}$, the **better** model performance is.
- An **absolute** measure of the **goodness of fit**.
- Easy to manipulate mathematically ("**well behaved**").
- **Sensitive** to **outliers**. Punishes large errors more.
- Actual **range** depends on the range of the **actual values**.
  - It can only be used to **compare** models on the **same data**, otherwise MSE is not very informative.

Root Mean Squared Error (RMSE): the **square root** of **MSE**.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- **MSE values** can be **too big** to compare easily, and the range of the **error** is **bigger** than the range of the **predicted values**.
- Taking a square root corrects those issues.
- **Everything else said for MSE holds with RMSE**.

R-squared (R^2), or coefficient of determination: the **ratio** between the **variance** explained by the model and **total variance**.

$$R^2 = \frac{\sum_{i=1}^{n}(\hat{y}_i - \bar{y})^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}, \quad \text{where} \quad \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

- $\bar{y}$ is the **average** of the actual values.
- $\sum_{i=1}^{n}(\hat{y}_i - \bar{y})^2$ is the **Regression Sum of Squares**: the sum of squares of the differences between **predicted** values and the mean of the **actual** ones.
  - Represents the **variance** in the real values **explained by the model**.
- $\sum_{i=1}^{n}(y_i - \bar{y})^2$ is the **Total Sum of Squares**: the sum of squares of the differences between **actual values** and their **mean**.
  - Proportional to the **variance** in the **actual** values ($n \times Var(Y)$).
- $\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$ is the **Residual Sum of Squares**.
  - Represents the **variance** in the real values **not explained by the model**.
- Values between $\mathbf{0}$ **and** $\mathbf{1}$ $\Rightarrow$ can be expressed as a percentage.
  - **Range independent** of the **data**.
  - Can be used to **compare** models where errors are measured in **different units** or **ranges**.
  - A value near 1 indicates **better** model.
- R^2 is related to the **correlation** between **real** and **predicted values**.
  - the square of Pearson's correlation coefficient in linear regression.
- Not **sensitive** to **outliers**.

Mean Absolute Error (MAE): arithmetic **mean** of the **magnitude** of the individual **errors** without taking account of their sign.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- Like **MSE**, but taking **absolute values** of residuals instead of squares.
- Values between $\mathbf{0}$ **and** $\boldsymbol{\infty}$.
  - The **closer** it is to 0, the **better** model performance is.
- It ignores the **direction** of the error, accounting for its **magnitude** only.
- Uses the same **scale** as the **data** being measured $\Rightarrow$ It can only be used to **compare** models, otherwise MAE is not very informative.
- Treats large and small errors equally, but **less sensitive to outliers** than MSE of RMSE.

- RMSE $>$ MAE
- **Not differentiable at zero**,
  - Not good for **optimization** algorithms.
  - It can be challenging to compute **gradients** with MAE.

Relative Squared Error (RSE): the **ratio** between the **Residual Sum of Squares** and the **Total Sum of Squares**.

$$RSE = \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

- It can be interpreted as the **ratio** between the **MSE** of our model and the MSE of a model which uses the **mean** of the actual values as the predicted value.
- Values between $\mathbf{0}$ **and** $\mathbf{1}$ $\Rightarrow$ can be expressed as a percentage.
  - **Range independent** of the **data**.
  - Can be used to **compare models** where errors are measured in **different units** or **ranges**.
  - The **closer** it is to $\mathbf{0}$, the **better** model performance is.

Relative Absolute Error (RAE): the **ratio** between the sum of absolute errors (**residuals**) and the differences between the observed **values** and their **mean**.

$$RAE = \frac{\sum_{i=1}^{n}|y_i - \hat{y}_i|}{\sum_{i=1}^{n}|y_i - \bar{y}|}$$

- Reflects how the **mean residual** relates to the **mean deviation** of the target function from its mean.
- Values between $\mathbf{0}$ **and** $\mathbf{1}$ $\Rightarrow$ can be expressed as a percentage.
  - **Range independent** of the **data**.
  - Can be used to **compare models** where errors are measured in **different units** or **ranges**.
  - The **closer** it is to $\mathbf{0}$, the **better** model performance is.
- **Not sensitive** to **outliers**.

Mean Absolute Percentage Error (MAPE): the arithmetic **mean** of the **relative errors**.

$$MAPE = \frac{1}{n} \sum_{i=1}^{n} \frac{|y_i - \hat{y}_i|}{y_i}$$

- Popular measure in the evaluation of **forecasting** methods in statistics, also used as **loss function** in regression problems.
- Computation is **problematic** if $\boldsymbol{y_i = 0} \Rightarrow$ used when the quantity to predict is known to remain way above zero.
- Values between $\mathbf{0}$ **and** $\mathbf{1} \Rightarrow$ can be expressed as a percentage.
  - **Range independent** of the **data**.
  - Can be used to **compare models** where errors are measured in **different units** or **ranges**.
  - The **closer** it is to $\mathbf{0}$, the **better** model performance is.

4.2 Evaluation of ML algorithms

Once we know the different metrics we can use to assess the performance of ML models, now we can focus out attention on how to conduct the evaluation of learning algorithms. In particular:

1. The different sources of **variability** in results, and how to treat them.
2. How to **partition a dataset** to generate training and test datasets.
3. How to check differences in performance via **hypothesis testing**.

4.2.1 Sources of variability in testing

When we evaluate the performance of learning algorithms, we either:

1. compare **different algorithms** $\Rightarrow$ to find which one is better for a given problem.
2. compare the same algorithm with **different parameters** $\Rightarrow$ to find which configuration is more adequate.

In order to have meaningful measurements (of accuracy, error rate, ...) to compare algorithms, we have to **minimize** the sources of **variability** (randomness):

1. **Determinism** of algorithms.
   - **Deterministic** algorithms (SVM, k-NN, ...) always produce the **same model** from the same training dataset.
     - They can be evaluated just **once** on the same datasets.
   - **Non-deterministic** algorithms (ANNs, bagging, ...) can produce **different models** from the same training dataset.
     - We cannot trust the results of a single execution (model).
     - There will be **variations** between models due to the non-determinism (built-in **randomness**) in the **learning algorithm**.
2. **Variability** in the data.
   - If we **randomly partition** the original dataset into a training and testing sets, we introduce another **source of variability**.

- A different partition could give us different results.
- **Unstable** algorithms will produce different models and results even with **small changes** in the training and testing datasets.

How to **minimize** both sources of variability:

- **Repeat** the experiment (training and testing) generating several models.
- **Average the results** of these models (accuracy, error rate, ...) $\Rightarrow$ the arithmetic mean can be used as a single measurement.

1. If we have a **fixed** split of the data (typical of big datasets).
   (a) Training/test examples specified beforehand and cannot be changed.
   (b) If the learning algorithm is **deterministic** $\Rightarrow$ run it just **once**.
      - The training data is used to train a model.
      - Test data is used to evaluate the model.
   (c) If the learning algorithm is **non-deterministic** $\Rightarrow$ run it **many times** (up to 50-100) and **average** results.
   (d) In both cases, we have a **single value** per learning algorithm for the given data.
   (e) In many applications there are datasets with **fixed splits** that are used to **compare** performances of learning algorithms.
2. If training/test data come form a **random partition** of the original dataset (usual for small datastets).
   (a) An initial set of examples, with no indication which ones will be used for training or testing $\Rightarrow$ **randomly** split into training/test datasets several times.
   (b) If the learning algorithm is **deterministic** $\Rightarrow$ run it **once** per split.
   (c) If the learning algorithm is **non-deterministic** $\Rightarrow$ run it **many times** (up to 50-100) with each partition of the data.
      - Results (accuracy, error rate, ...) on the test set are **averaged**.
      - That average (a single value per metric) will be the test result for the current split of the data.
   (d) Results obtained on all splits are **averaged** $\Rightarrow$ average is used for **comparison** purposes.

4.2.2 Dataset partition

In a partition of the original data, a decision has to be made about the **size of the test set**.

- If the number of instances is **low** (thousands or tens of thousands) $80\%/20\%$, $85\%/15\%$ or similar proportional splits are typical.
  - If a **validation** dataset is necessary (more on that later) it would have the same size of the test set: $60\%/20\%/20\%$ or $70\%/15\%/15\%$ or similar.
- If the number of instances is **high** (millions or higher), we can go to **smaller test sets**: $99\%/1\%$, $99.5\%/0.5\%$, $\ldots$.
  - With **validation** dataset: $98\%/1\%/1\%$, $99\%/0.5\%/0.5\%$ or similar.
- The number of instances in the **test** set will **similar** in both cases.
- It is more important that those **instances** are **representative**.

There are several techniques to partition a dataset into training and test sets. We will discuss them and their impact in the evaluation process:

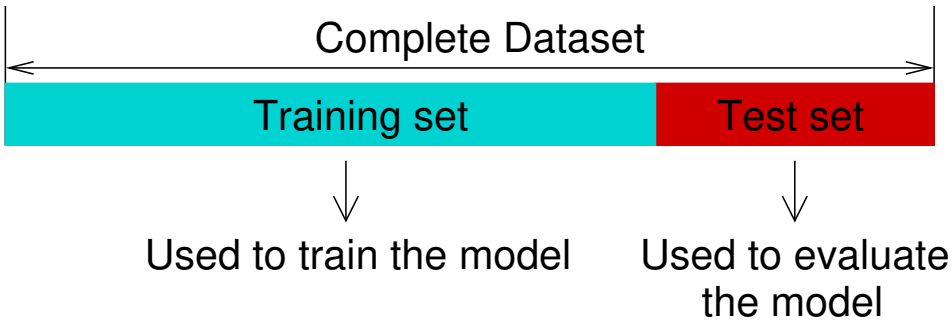
- Hold out.
- Random subsampling.
- Bootstrapping.
- (stratified) k-fold cross validation.
- Leave-p-out cross validation.
- Leave-1-out cross validation.

Goal (of these techniques): to **minimize the variability** arisen from the **random split** of instances into training and test sets.

- **Deterministic** learning algorithms: that is the only source of variability.
  - A **single experiment** in each data partition is **enough**.
- **Non-deterministic algorithms:** additional source of variability.
  - A **single experiment** is **not** representative.
  - Each experiment will give a diferent result, even on the same datasets.
  - It is necessary to **repeat** the whole process again.
    - with the **same partition** in training and test data,
    - **averaging** results.
  - Additional tests can concentrate in minimizing the **variability** due to the **algorithm**.

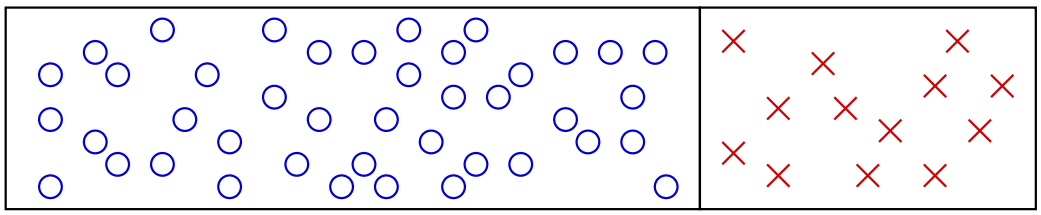
Basic Partition Techniques

Hold out: **random split** of the dataset in two **disjoint subsets** (with no common instances): training and test.



- The **simplest** way of partitioning a dataset.
- It is considered a **pessimistic** evaluation.
  - The **training dataset** is **smaller** than the original dataset.
  - We may be **leaving out** particularly **useful instances** for model generalization out of the **training** data (e.g., instances that cover a particular region of the feature space).
  - We would probably get **better results** training with the **whole dataset**.
- Given that we are conducting a single experiment (in case of deterministic algorithms, at least) the results will be **misleading** if the **partition** is **skewed**.
  - Worst-case scenario: binary classification.

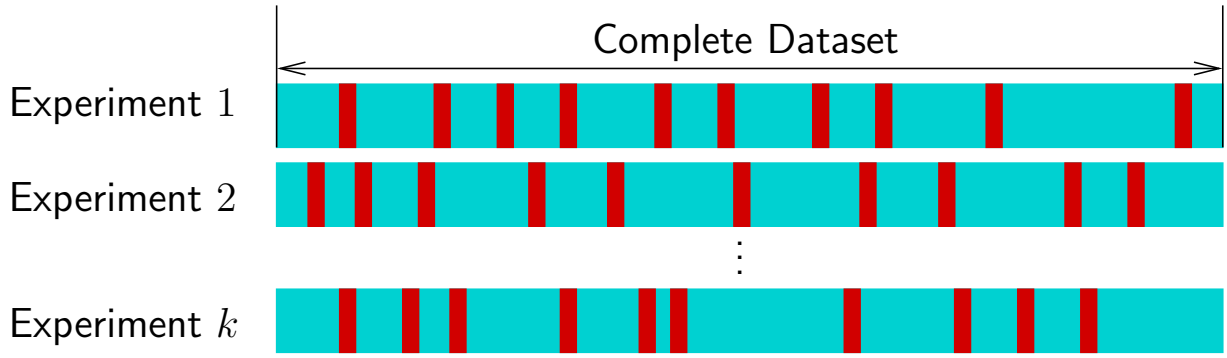
Extreme case: two classes, ○ in training set × in test set



  - **Results** are **dependent** on the **random splitting** of the data.
  - That source of **variability** is **not** being **minimized**.
  - **Random subsampling** ameliorates this problem by repeating the experiment with other random splits.

Random subsampling: **hold out**, repeated k **times**.

- We derive training datasets with **sampling w/o replacement**.
- The **results** of the k experiments are **averaged**.



- **Problem**: some instances may never be selected as part of a test set, while others may be selected several times.

Bootstrapping: same sampling technique used in **bagging**.

- **Training datasets** from **uniform sampling with replacement**.
  - After an instance is chosen is put back in the original dataset.
  - It is available to be sampled again.
- The **test set** will made of the instances not in the training set.
- **0.632 bootstrap**. Given a training size of size n.
  - n instances in the original dataset $\Rightarrow$ n instances in the training set.
  - Instances not in the training set go to the test set.
  - Probability of an instance being chosen for large enough values of n:

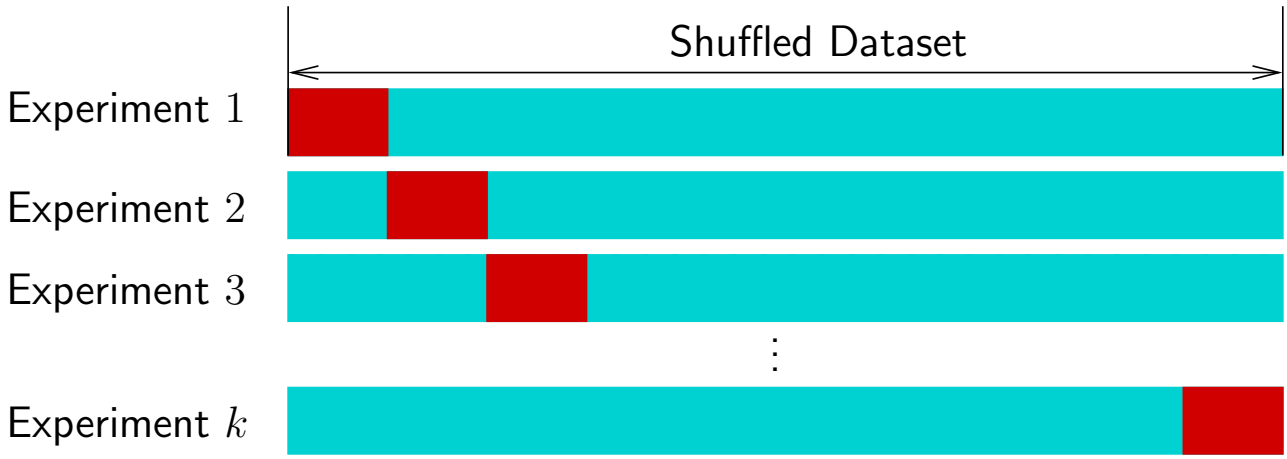
$$1 - \left(1 - \frac{1}{n}\right)^n \simeq 0.632$$

  - About $\mathbf{63.2}\%$ of the **examples** in the bootstrap $\Rightarrow$ **training set**. The remaining 36.8% will go in the test set.

Complete dataset	x_1, x_2, x_3, x_4, x_5	
Experiment 1	x_3, x_1, x_3, x_3, x_5	x_2, x_4
Experiment 2	x_5, x_3, x_5, x_2, x_1	x_4
	$\vdots$	
Experiment k	x_4, x_2, x_4, x_4, x_2	x_1, x_3, x_5
	Training Sets	Test Sets

$\boldsymbol{k}$**-fold cross validation**: a **popular approach** designed to solve the problem of random subsampling.

1. The original dataset is randomly **shuffled**.
2. The shuffled dataset is divided in $\boldsymbol{k}$ **disjoint** subsets $\{D_1, D_2, \ldots, D_k\}$.
   - The first $\lceil n/k \rceil$ examples on the shuffled dataset are set as D_1,
   - The following $\lceil n/k \rceil$ examples are set as D_2,
   - and so on.
3. Thus, all **subsets** $\boldsymbol{D_i}$ will be of the **same size**.
   - If n is not a multiple of $k \Rightarrow D_k$ may be slightly smaller.
4. The learning algorithm is used to train and test $\boldsymbol{k}$ **models**:
   - One experiment (training and test) for each fold i.
   - $\boldsymbol{D_i}$ is used as **test** set and the **remaining** $\boldsymbol{k-1}$ subsets are used as the **training** set.
5. The final result of all tests will be the arithmetic **mean** of the k results.
   - Results are **slightly pessimistic**.



- k-fold cross validation has similarities to **random subsampling**:
  - Main difference: k-fold cross validation guarantees that **all instances** are used in training and testing.
  - **Better** for **small datasets**.
- Important **design choice**: the value of $\boldsymbol{k}$.
  - It will determine how **precise** are the results obtained and the **computational cost** of the experiment.

- If $\boldsymbol{k}$ is **high**:
  * Results will be very proximate to the real values $\Rightarrow$ **less bias**.
  * **Variance** of those results will be **high**.
  * Computational **cost** will be **high** (many training/testing cycles).
  * **Extreme case**: **leave-1-out** cross validation ($k = n$).
- If $\boldsymbol{k}$ is **small**:
  * Results will be less proximate to the real values $\Rightarrow$ **more bias**.
  * **Variance** will be **low**.
  * Computational **cost** will be **low**.
- In practice, the choice of k depends on original **dataset size**.
  * **Big** datasets: even a **3-fold cross** validation will obtain precise results.
  * **Very** small datasets: a **leave-1-out** strategy may be advisable, in order to have as much data as possible in the training set.
- A **usual value** is $\boldsymbol{k} = 10 \Rightarrow$ **10-fold cross validation**.
  * 5-fold validation is less popular, but also used, especially with big datasets.

Stratified $\boldsymbol{k}$-fold cross validation: a variant of k-fold cross validation.

- Instances are distributed among D_i subsets so there are the same number of examples of each class in each subset D_i.
  - The **proportion** of instances of **each class** is the **same** (or very similar) as in the original dataset.
  - Particularly **useful** with **unbalanced datasets**.

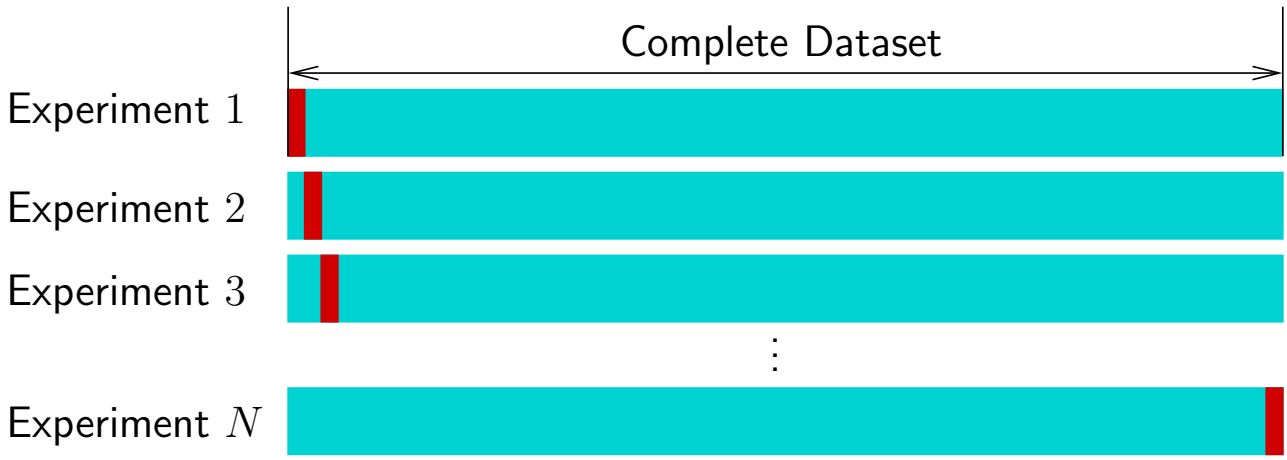
Leave-$\boldsymbol{p}$-out: a extreme case of cross validation.

1. Given a dataset of size n, a **sample of $\boldsymbol{p}$ instances** is drawn from the original dataset **w/o replacement** $\Rightarrow$ test set.
2. The remaining $n - p$ instances will be used as the training set.
3. We fit a model to the training data and estimate its performance in the test data.
4. The process is repeated for **all possible samples of size $\boldsymbol{p}$**.
5. The results of all experiments are **averaged**.

- **Extremely costly** in computational terms $\Rightarrow$ number of possible samples of size p (number of iterations) grows combinatorially with n.
  - Useful in **small datasets**.
  - Results tend to be more reliable than other cross validation techniques.
- An **exhaustive** method of cross validation: every single possibility of dividing the dataset into training and testing subsets is tried.

Leave-1-out cross validation: a technique better used with small datasets.

- It can be understood as:
  - A particular case of $\boldsymbol{k}$**-fold cross** validation, with $\boldsymbol{k = n}$.
  - A particular case of **leave-$\boldsymbol{p}$-out** validation, with $\boldsymbol{p = 1}$.
- For an original dataset of $\boldsymbol{n}$ **instances** we train and test $\boldsymbol{n}$ **models**.
- In each experiment $n - 1$ instances are used to train the model and the remaining instance is used to test it.
- Useful with **small datasets**.
- Computational **cost** grows with n, though not as fast as with leave-p-out cross validation.
- Another **exhaustive** method of cross validation.



Validation datasets and hyperparameter tuning

Most learning algorithms have **hyperparameters**, whose values are used to control the learning process, and can be set by the user.

E.g., topology in ANNs, type of kernel and value of C in SVMs.

One choice is to use hyperparameter values already seen in the **literature**.

But, in many cases, we want to **optimize** them as part of our experiments,

- to **maximize** the **performance** of our models,
- in the particular **problem** we are trying to solve.
- A process known as **hyperparameter optimization** or **hyperparameter tuning**.

In order to perform hyperparameter tuning we need to:

- train and test models with different parameter values, and
- choose the model with best performance (along with its hyperparameter values).

But we want to report the **unbiased performance** of the chosen model, measuring it on a **dataset** not used in the training or tuning processes.

Therefore, to tune the hyperparameters of an algorithm while evaluating its performance, we are going to partition de data in **three** disjoint datasets:

- **Training dataset**: used for the training of the different parametrizations of the learning algorithm.
- **Validation dataset**: used to determine the optimal hyperparameters.

  E.g., in algorithms with iterative training methods (such as stochastic gradient descent) the validation loss can be used to conduct **early stopping** to avoid overfitting.
- **Test dataset**: used to measure the performance of the model with the optimal hyperparameters.

(**Note on terminology**: the **validation** dataset is sometimes called the **development** dataset)

Separate validation and test sets are used because:

- The **validation** dataset is used in the **optimization** process (selection of the best model).
- An **evaluation** of the best model on the validation data would be **biased** (optimistic).
- We need a **test dataset** for an **unbiased** evaluation.
- **No tuning** is to be performed after the model with the chosen hyperparamaters is evaluated on the test set!

Experimental procedure:

1. **Split** the data in training, validation and test datasets (using cross validation, bootstrap, ...)
2. **Select** the **hyperparameters** of the training algorithm.
3. **Fit** a model to the **training** dataset.
4. **Evaluate** the model on the **validation** dataset.
5. Repeat **steps 2-4** using different configurations and initial parameters.
6. Identify the **best model**.

   Two options:

   (a) Select the best model as the **final one**.

   (b) **Train** a final model with the **same hyperparamaters** of the best model on the training and validation data.
7. **Evaluate** the final model on the **test** dataset.

Step 6: Choice of (a) or (b) depends on the kind of **learning algorithm** used ⇒ deterministic or non-deterministic.

Example, option (a): Artificial Neural Networks (ANNs).

- Different hyperparameter values ⇒ e.g., network topologies.
- Training is **not deterministic** ⇒ each topology has to be trained and validated many times (steps 3-4 are repeated).
- The ANN with the lowest validation error in all topologies is chosen, and evaluated in the **test dataset**.

- We do **not train again** with the same topology on the combination of training and validations datasets.
  - The resulting dataset would have to be split again in training and validation datasets.
  - The use of **a validation dataset is intrinsic to ANN training**.

Example, option (b): Support Vector Machines (SVMs).

- Different hyperparameter values $\Rightarrow$ e.g., kernel type, parameters and C.
- Training is **deterministic**
  - Only one model is trained per parametrization.
- The best performing model on the validation dataset (and its hyperparameter settings) is chosen.
- A **final model** with the same hyperparameters as the previously chosen is fit to the **training+validation dataset**.
  - **New instances** used in training $\Rightarrow$ **additional information**.
  - The new model is expected to have **better performance**.
- The final model is evaluated on the **test dataset**.

4.2.3 Comparing learning algorithms

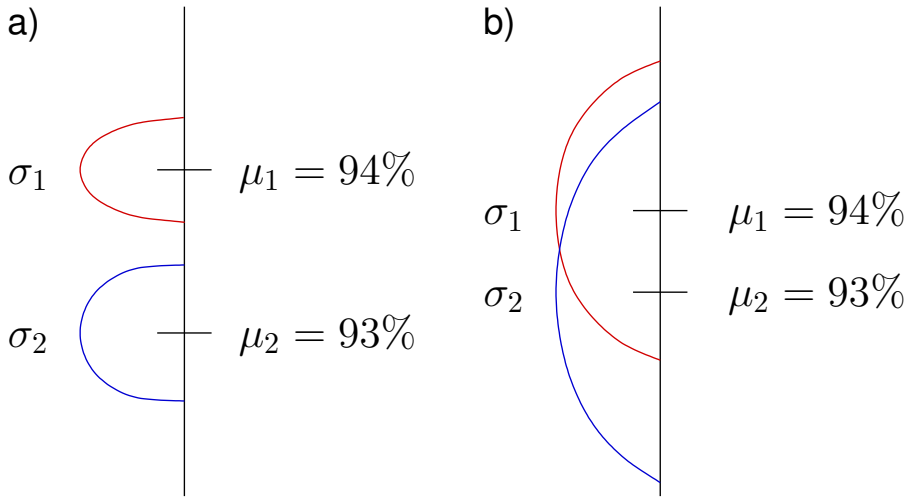
(or the same algorithm with different parameters)

In the previous discussion about sources of variability, and data partition, the preferred solution to assess the performance of a learning algorithm has been

1. to conduct **more than one test**, in different data partitions, in the same data partition or both, and
2. **to average** the results obtained with the metric(s) of our choice.

But that **may not be good enough** if we want to compare the performances of different learning algorithms (or different parametrizations of the same algorithm).

- Let's say we have two algorithms with average accuracies of 93% and 94%, but very different standard deviations.



  - A hypothesis test would consider the results in a) as **significantly different**.
  - A hypothesis test may **not** consider the difference between the averages in b) as **significant**.
- Results of two algorithms too close $\Rightarrow$ a **contrast hypothesis** will be necessary to determine if their performances are statistically dissimilar.

Let's start with the **simplest case**. We want to compare the performance of **two algorithms** (or parametrizations of the same algoritm).

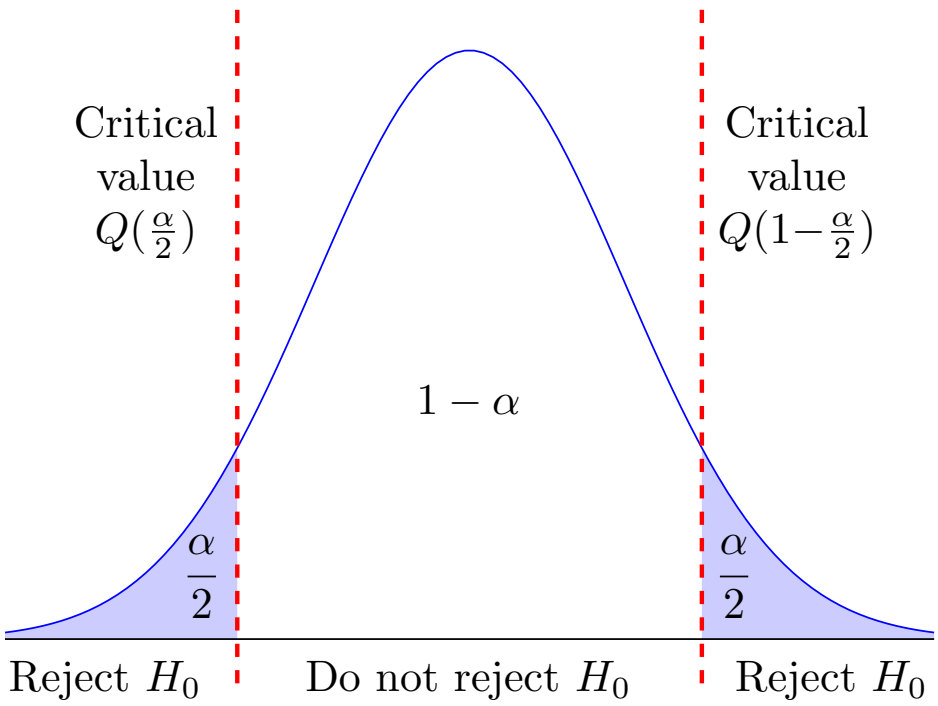
- We have a certain number of measurements of a given metric per algorithm: accuracy, error rate, F1, ...

- E.g., k-fold cross validation.
  - We have a $\boldsymbol{k}$ **accuracies** for each type of model, computed on the same datasets.

- With that information, we can proceed with a contrast hypothesis.

Option 1: critical values

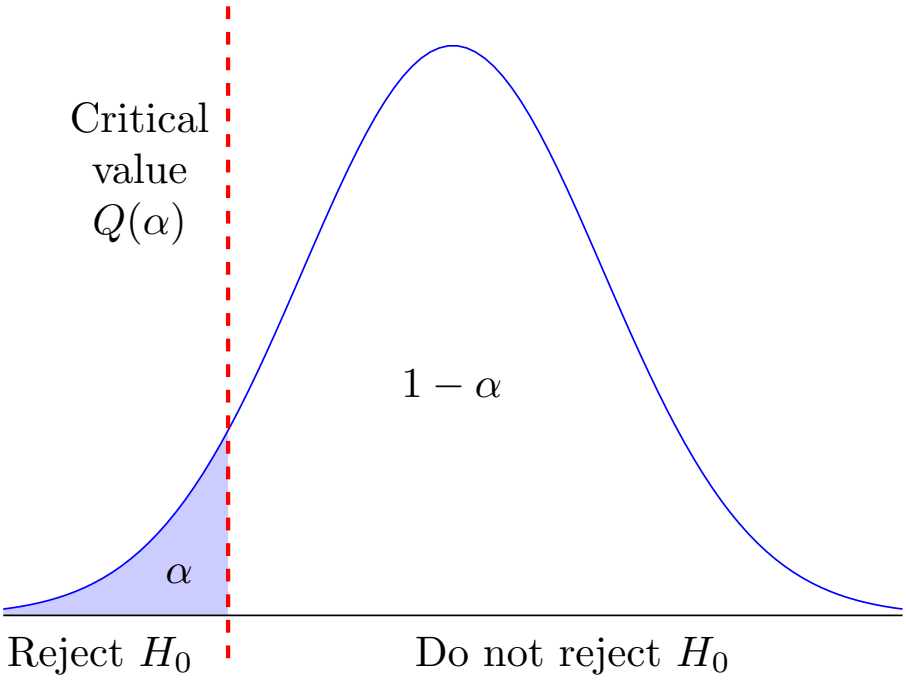
1. Define the **null hypothesis** H_0 (no difference between measurements).
2. Select a test that is appropriate to establish the validity of H_0, and state the relevant **test statistic** T.
   - T is a quantity derived from the sample.
3. Select a **significance level** α.
   - Probability of reject H_0 being true $\Rightarrow$ typically 0.01, 0.05 (1%, 5%).
4. Determine the **critical value** determining the **rejection region**.
   - The set of values of the test statistic for which the null hypothesis is rejected.
5. Compute the value of T from the **particular sample**.
   - If the value of T is in the rejection region $\Rightarrow$ reject H_0.
   - Otherwise $\Rightarrow$ do not reject H_0.



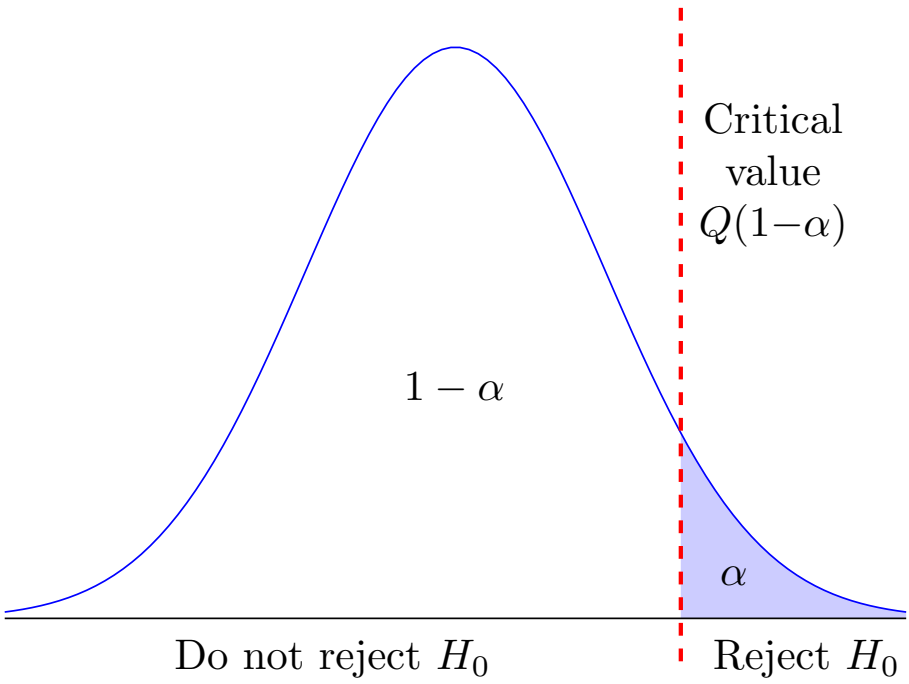
Two-sided test

Three options for the **rejection region**:

- **Two-sided test**: H_0 rejected if $T \in \left[Q(\frac{\alpha}{2}),\ Q(1-\frac{\alpha}{2})\right]$ (prev. figure).
- **Left-tailed test**: H_0 rejected if $T \in (-\infty,\ Q(\alpha)]$
- **Right-tailed test**: H_0 rejected if $T \in [Q(\alpha),\ \infty)$



Left-tailed test



Right-tailed test

Example: Paired samples t-test (Student's t-test)

1. We have n **measurements** of performance (e.g., accuracies, F-scores, ...) of two algoritms on the same datasets (e.g., from a cross validation).
   - μ_1 and μ_2 are the **averages** of the performances.
   - **Null hypothesis** H_0: $\mu_1 - \mu_2 = 0$
2. Compute the **test statistic** T for paired samples.

   $$T = \frac{\bar{d}}{\sigma/\sqrt{n}} = \frac{\sqrt{n}\,\bar{d}}{\sigma} \qquad \bar{d} = \frac{1}{n}\sum_{i=1}^{n} d_i \qquad \sigma = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(d_i - \bar{d})^2}$$

   - d_i: performance difference between both algorithms in observation i.
   - $\bar{d}$: average of the differences.
   - σ: the standard deviation of the differences.

   T follows a ***t*-distribution** with $n-1$ degrees of freedom under H_0.
3. Search the **critical value** in a table of selected values for the t-distribution.

   Based on the $n-1$ **degrees** of freedom and **significance level** α.
   - **two-sided**: $\mu_1 < \mu_2$ or $\mu_1 > \mu_2 \Rightarrow \left[Q_{t,n-1}(\frac{\alpha}{2}),\ Q_{t,n-1}(1-\frac{\alpha}{2})\right]$
   - **left-tailed**: $\mu_1 < \mu_2 \Rightarrow (-\infty,\ Q_{t,n-1}(\alpha)]$
   - **right-tailed**: $\mu_1 > \mu_2 \Rightarrow [Q_{t,n-1}(1-\alpha),\ \infty)$

   The distribution is symmetric around 0: $Q_{t,n-1}(\alpha) = -\,Q_{t,n-1}(1-\alpha)$

   Typical tables will give only the positive critical values (right-tailed).
4. If T is in the rejection region, reject $H_0 \Rightarrow$ differences in performance are statistically significant (not due to chance).
   Otherwise, the differences are **not** statistically significant.

E.g., let's say that $n = 10$, $\alpha = 0.05$ and $T = 1.874$, and two-sided.

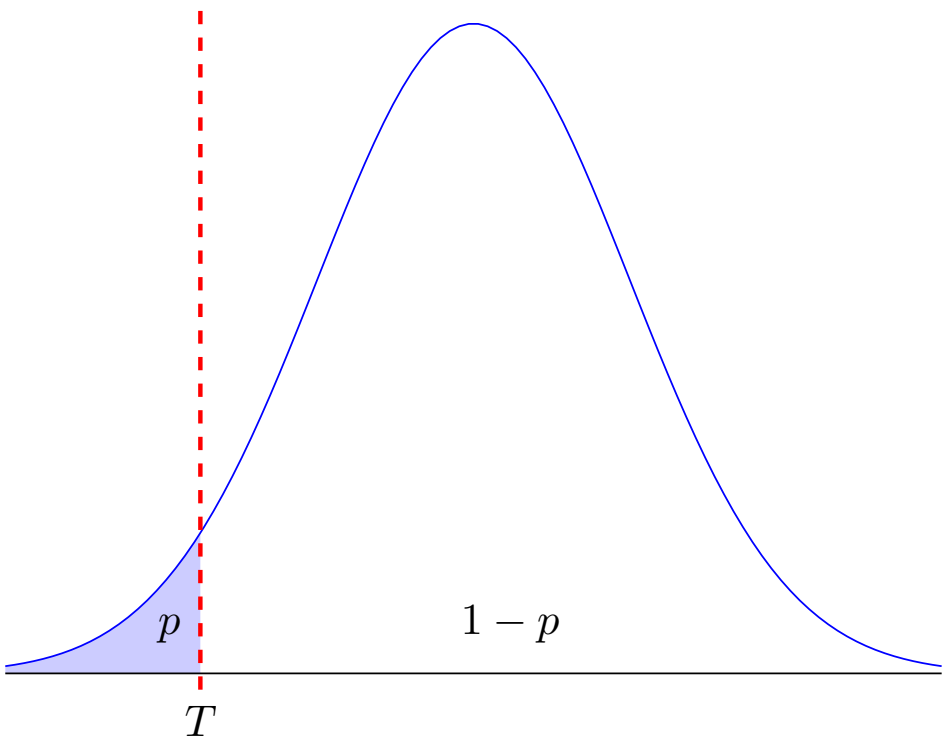
- Get table of selected values for the t-distribution (e.g., this one)
  - Row 9: distribution with 9 degress of freedom.
  - Column $\alpha = 0.05$ and two tails.
  - Critical value: 2.262.
- Since $1.874 \in [-2.262, 2.262]$ (nonrejection region) H_0 is **not** rejected.

  The difference in performance is **not** statistically significant.

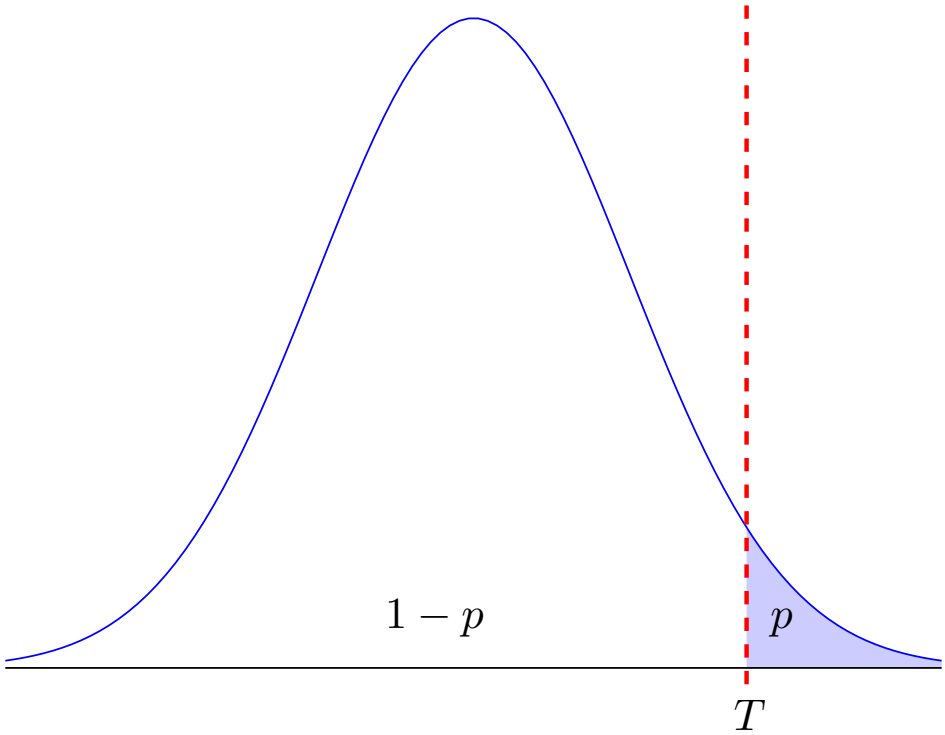
Option 2: p-values

1. Define the **null hypothesis** H_0 (no difference between measurements).
2. Select a test that is appropriate to establish the validity of H_0, and state the relevant **test statistic** T.
   - T is a quantity derived from the sample.
3. Select a **significance** level α.
   - Probability of reject H_0 being true $\Rightarrow$ typically 0.01, 0.05 (1%, 5%).
4. Calculate T for this sample and, from it, the **p-value**:
   - **Probability** of obtaining a real-valued test statistic T at least as extreme as the one actually obtained.
   - Under the **assumption** that H_0 is correct.
5. Compare the p-value and the significance level α.
   - If $p \leq \alpha \Rightarrow$ reject H_0.
   - If $p > \alpha \Rightarrow$ do not reject H_0.

How to compute p: same three cases that with critical values.



Left-tailed test



Right-tailed test

In the **two-sided** test, the p-value is usually computed as two times the value of the right-tailed test.

In both options, the **null hypothesis** will typically indicate that the **difference in performance** values of the algorithms being compared is **not statistically significant**.

- If we are comparing a new and brilliant solution of our own to a baseline, we really want H_0 to be rejected.

Statistical software packages usually compute both T and p-values for several well-known statistical tests, given the sample data.

- In practice, you will use p-values most of the time, instead of critical values.

Now that we have an understanding of how statistical testing works, we will finish with a brief explanation of the most commonly used tests.

- Pair-wise comparison of results.
- Comparison of multiple algorithms.

Two sets of measurements

We compare the performance **measurements** (accuracy, F-score, ...) of **two** types of models (or of the same type with different hyperparameters) on the same tests sets:

- k-fold cross validation, random subsambling, bootstrapping, ...
- or just different datasets.

A) (Paired samples) Student's t-test: a t-test is any statistical hypothesis test in which the test statistic T follows a Student's t-distribution under the null hypothesis.

A paired (or matched) samples t-test determines if the **means** of two sets of measurements on the same n units are **different**.

- Null hypothesis (H_0): $\mu_1 - \mu_2 = 0$

For the test to work, the following **assumptions** have to hold:

- Both populations follow a **normal distribution** (**parametric** test).
  - Slight deviations of this assumption may be tolerated, in particular if the number of measurements is large enough ($n \geq 30$).
- Both populations present the **same variance**.

Having $n \geq 30$ is often advised, especially with random subsampling.

More robust variations:

- **5x2cv**: 5 rounds of 2-fold cross validation.

$$T = \frac{d_{11}}{\sqrt{\frac{1}{5}\sum_{i=1}^{5}\sigma_i^2}} \qquad \bar{d}_i = \frac{1}{2}\sum_{j=1}^{2} d_{ij} \qquad \sigma_i^2 = \sum_{j=1}^{2}(d_{ij} - \bar{d}_i)^2$$

  - d_{ij}: performance difference between both algorithms, round i, fold j.
  - $\bar{d}_i$: average of the differences in round i.
  - σ_i^2: the variance of the differences in round i.

  following a t distribution with 5 degrees of freedom.

- **Combined 5x2cv F test**: again 5 rounds of 2-fold cross validation.

$$F = \frac{\sum_{i=1}^{5}\sum_{j=1}^{2} d_{ij}^2}{2\sum_{i=1}^{5}\sigma_i^2} \qquad \bar{d}_i = \frac{1}{2}\sum_{j=1}^{2} d_{ij} \qquad \sigma_i^2 = \sum_{j=1}^{2}(d_{ij} - \bar{d}_i)^2$$

  under a F-distribution with 10 and 5 degrees of freedom.

- **Corrected paired student t-test**: used with random subsampling or k-fold cross validation.

$$T = \frac{\bar{d}}{\sqrt{\left(\frac{1}{n} + \frac{n_1}{n_2}\right)\sigma^2}} \qquad \bar{d} = \frac{1}{n}\sum_{i=1}^{n} d_i \qquad \sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(d_i - \bar{d})^2$$

  where n_1 and n_2 are the number of instances in the training and test sets, respectively, $\bar{d}$ is the average of differences and σ their variance.

B) Wilcoxon signed-rank test: a **non-parametric** alternative to the paired samples t-test.

- Null hypothesis (H_0): $M_1 = M_2$ (same median in both distributions).
- Being non-parametric, there is no assumption that the samples follow a normal distribution.

When to use paired samples t-test and Wilcoxon signed-rank test?

1. We should use the ***t*-test** if its **assumptions** (normal distribution, equal variance) hold.
   - It has a bigger probability of rejecting H_0 is it is false.
2. **Otherwise**, the **Wilcoxon** signed-rank test is more reliable.
   - It does not assume any distribution for the data.
   - It is more reliable with atypical cases.
   - If the distribution followed by the data is not known, it is the one that should be used.

Whichever test we are using, its results can tell us which of the two competing models to use.

1. If the **null hypothesis** is **not rejected**, the difference in performance between the models is not statistically significant.
   We should choose the **simplest** one (lowest number of parameters).
2. **Otherwise** we should choose the **best performing** one.

Multiple sets of measurements

If we want to compare **more than two** learning algorithms, a **pairwise comparison** using Student or Wilcoxon is **not advisable**:

- The probability of an erroneous rejection of H_0 increases with the number of tests.
- **Lots of people do pairwise testing anyway.**

One possible **solution** is to use **multiple test correction** techniques:

- Pairwise t-tests (or Wilcoxon signed-rank tests),
  with reduction of individual **significance levels** $\boldsymbol{\alpha}$,
  in order to reduce the probability of erroneously rejecting H_0.
- Reduction of α: Bonferroni correction, Šidák correction, ...
- **Problem**: if α is reduced too much, H_0 may be erroneously accepted in some pairwise tests.

The **recommended solution** is to use tests capable of finding if there is a statistically significant difference between the **averages** of **more than two groups** of data: Repeated Measures ANOVA and Friedman test.

1. If **no** significant **differences** are found.
   - We choose again the **simplest** learning algorithm.
2. If a significant **difference** is found.
   - At least one of the means is statistically different from the others.
   - We can find which one(s) using a post-hoc test for differences between multiple groups: Fisher's LSD, Tukey's HSD, Scheffé, Nemenyi, Bonferroni, Šidák, ...
   - **Best** learning algorithm, or the **simplest among the best**, is chosen.

A) Repeated measures ANOVA: **parametric** test to compare the **averages** of different groups of data (in our case performance measurements).

- ANOVA = ANalysis Of VAriance
- Repeated measures ANOVA $\Rightarrow$ paired data
- Null hypothesis (H_0): $\mu_1 = \mu_2 = \ldots = \mu_m$ (same averages)

Assumptions:

- Parametric test: all samples are **normally distributed**.
- Sphericity: equal **variances** among the **differences** between all possible pairs of measurements.
- Samples are drawn from a **continous variable**.

The test can operate even in presence of slight deviations from the first two.

B) Friedman test: **non-parametric** test to compare the **averages** of different groups of data.

- Null hypothesis (H_0): $M_1 = M_2 = \ldots = M_m$ (same medians)

Assumptions:

- Samples are drawn from a **continous variable**.

When to use each test?

- **ANOVA** should be used when its **assumptions** (normal distribution, sphericity) are met.
  - More powerful than Friedman test.
- **Otherwise**, **Friedman** probably better.

Paired vs unpaired samples

All previous tests assume matched/paired samples.

- A less common scenario is to have measurements of several models on different test sets.

Each one of the previous tests has a counterpart in this scenario:

- Paired samples t-test $\Rightarrow$ t-test for unpaired samples.
- Wilcoxon signed-rank test $\Rightarrow$ Mann-Whitney U.
- Repeated measures ANOVA $\Rightarrow$ ANOVA.
- Friedman test $\Rightarrow$ Kruskal-Wallis test.